МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных технологий

Отчет по практической работе №3

по дисциплине «Тестирование и верификация ПО»

Тема практической работы: «Подходы для разработки программного обеспечения»

Выполнил:
Студент группы ИКБО-31-24 Жильцова С.О. /

Проверил:
Доцент кафедры МОСИТ,
кандидат технических наук, доцент
Петренко А.А. /

18/02/26

Москва 2026

Цель работы: изучить и применить различные подходы к разработке программного обеспечения, основанного на тестировании, для повышения качества, надёжности и поддерживаемости кода.

Задачи:

1) изучить теоретические основы методологий тестирования: TDD, ATDD, BDD и SDD;

2) исследовать преимущества и недостатки каждого подхода;

3) реализовать практические примеры для каждого метода;

4) проанализировать влияние интеграции тестирования на архитектуру и качество программного продукта;

5) подготовить итоговый отчёт с выводами и рекомендациями по интеграции подходов в современные процессы разработки.

Вариант: 75

75. Расписание фитнес-классов

Функции – создание расписания, регистрация участников, отмена занятия.

TDD – тесты для функций создания и обновления расписания.

ATDD – сценарии для проверки корректности регистрации.

BDD – сценарий «Given класс запланирован, When участник регистрируется, Then он отображается в списке».

SDD – таблицы с примерами расписания и регистраций.

Ход работы:

Этап №1. Реализация с помощью TDD (Test-Driven Development).

Были написаны юнит-тесты для функций создания и обновления расписания (листинг 1). Для написания тестов использовалась библиотека unittest языка Python.

Листинг 1. Юнит-тесты для TDD

```
import unittest
```

```
from datetime import datetime, timedelta
from fitness import FitnessSchedule, FitnessClass, RegistrationError


class TestFitnessScheduleTDD(unittest.TestCase):

    def setUp(self):
        self.schedule = FitnessSchedule()
        self.future_date = datetime.now() + timedelta(days=3)
        self.fitness_class = FitnessClass(
            title="Йога",
            trainer="Иванова А.",
            datetime_start=self.future_date,
            capacity=10
        )

    def test_add_class(self):
        """Создание занятия добавляет его в расписание."""
        self.schedule.add_class(self.fitness_class)
        self.assertIn(self.fitness_class, self.schedule.classes)

    def test_register_participant(self):
        """Регистрация участника на занятие."""
        self.schedule.add_class(self.fitness_class)
        self.schedule.register(self.fitness_class.title, "Петров П.")
        self.assertIn("Петров П.", self.fitness_class.participants)

    def test_register_duplicate_raises(self):
        """Повторная регистрация того же участника вызывает ошибку."""
        self.schedule.add_class(self.fitness_class)
        self.schedule.register(self.fitness_class.title, "Петров П.")
        with self.assertRaises(RegistrationError):
            self.schedule.register(self.fitness_class.title, "Петров П.")

    def test_register_full_class_raises(self):
        """Регистрация в заполненный класс вызывает ошибку."""
        small_class = FitnessClass(
            title="Пилатес",
            trainer="Сидорова М.",
            datetime_start=self.future_date,
            capacity=1
        )
        self.schedule.add_class(small_class)
        self.schedule.register("Пилатес", "Петров П.")
        with self.assertRaises(RegistrationError):
```

```
            self.schedule.register("Пилатес", "Смирнов С.")

    def test_cancel_class(self):
        """Отмена занятия удаляет его из расписания."""
        self.schedule.add_class(self.fitness_class)
        self.schedule.cancel_class(self.fitness_class.title)
        self.assertNotIn(self.fitness_class, self.schedule.classes)

    def test_register_to_unknown_class_raises(self):
        """Регистрация на несуществующее занятие вызывает ошибку."""
        with self.assertRaises(RegistrationError):
            self.schedule.register("Несуществующее", "Петров П.")


if __name__ == "__main__":
    unittest.main()
```

В соответствии с методологией TTD на втором этапе был реализован код, представленный в листинге 2.

Листинг 2. . Код программы

```
from datetime import datetime


class RegistrationError(Exception):
    """Ошибка при регистрации участника."""
    pass


class FitnessClass:
    def __init__(self, title: str, trainer: str, datetime_start: datetime, capacity: int):
        self.title = title
        self.trainer = trainer
        self.datetime_start = datetime_start
        self.capacity = capacity
        self.participants = []

    def __repr__(self):
        return f"FitnessClass({self.title!r}, {self.trainer!r})"


class FitnessSchedule:
    def __init__(self):
        self.classes = []
```

```
    def add_class(self, fitness_class: FitnessClass) -> None:
        self.classes.append(fitness_class)

    def get_class(self, title: str) -> FitnessClass:
        for c in self.classes:
            if c.title == title:
                return c
        raise RegistrationError(f"Занятие {title!r} не найдено")

    def register(self, title: str, participant: str) -> None:
        fitness_class = self.get_class(title)
        if participant in fitness_class.participants:
            raise RegistrationError(f"{participant} уже записан на {title!r}")
        if len(fitness_class.participants) >= fitness_class.capacity:
            raise RegistrationError(f"На занятие {title!r} нет свободных мест")
        fitness_class.participants.append(participant)

    def cancel_class(self, title: str) -> None:
        fitness_class = self.get_class(title)
        self.classes.remove(fitness_class)
```

Запустив файл с тестами, можно убедиться, что все они успешно пройдены (рисунок 1).

```
Testing started at 14:51 ...
Launching pytest with arguments C:\Users\Sofiya\PycharmProjects\caesarCipher\test_tdd.py -

============================= test session starts =============================
collecting ... collected 6 items

test_tdd.py::TestFitnessScheduleTDD::test_add_class PASSED               [ 16%]
test_tdd.py::TestFitnessScheduleTDD::test_cancel_class PASSED            [ 33%]
test_tdd.py::TestFitnessScheduleTDD::test_register_duplicate_raises PASSED [ 50%]
test_tdd.py::TestFitnessScheduleTDD::test_register_full_class_raises PASSED [ 66%]
test_tdd.py::TestFitnessScheduleTDD::test_register_participant PASSED    [ 83%]
test_tdd.py::TestFitnessScheduleTDD::test_register_to_unknown_class_raises PASSED [100%]

============================== 6 passed in 0.02s ==============================

Process finished with exit code 0
```

Рисунок 1. Тестирование

Этап №2. Реализация с помощью ATDD (Acceptance Test-Driven Development).

Реализованы приёмочные тесты для сценариев использования приложения, которые были согласованы с конечными пользователями.

Сценарий 1. Создание занятия

Предусловие: пользователь открывает раздел «Расписание».

Действия:

1) пользователь вводит название занятия (например, «Йога»), имя тренера (например, «Иванова А.»), дату и время занятия, вместимость (максимальное количество человек, например, 10)

2) нажимает «Сохранить».

Ожидаемый результат: занятие «Йога» появляется в расписании.

Сценарий 2. Регистрация участника

Предусловие: в расписании есть занятие (например, «Йога» с вместимостью 10).

Действия:

1) пользователь вводит свое имя (например, «Петров П.») и нажимает «Записаться».

Ожидаемый результат: «Петров П.» отображается в списке участников занятия «Йога».

Сценарий 3. Отмена занятия

Предусловие: в расписании есть занятие (например, «Йога»).

Действия:

1) пользователь нажимает «Отменить занятие».

Ожидаемый результат: занятие исчезает из расписания.

Далее были реализованы приемочные тесты (листинг 3).

Листинг 3. Приемочные тесты ATDD

```
import unittest
from datetime import datetime, timedelta
from fitness import FitnessSchedule, FitnessClass, RegistrationError
```

```
class TestFitnessScheduleATDD(unittest.TestCase):
    """Приёмочные тесты."""

    def setUp(self):
        self.schedule = FitnessSchedule()
        self.tomorrow = datetime.now() + timedelta(days=1)

    def test_scenario_1_create_class(self):
        """Сценарий 1: создание занятия."""
        self.schedule.add_class(FitnessClass("Йога", "Иванова А.", self.tomorrow,
10))
        titles = [c.title for c in self.schedule.classes]
        self.assertIn("Йога", titles)

    def test_scenario_2_register_participant(self):
        """Сценарий 2: регистрация участника."""
        self.schedule.add_class(FitnessClass("Йога", "Иванова А.", self.tomorrow,
10))
        self.schedule.register("Йога", "Петров П.")
        yoga = self.schedule.get_class("Йога")
        self.assertIn("Петров П.", yoga.participants)

    def test_scenario_3_cancel_class(self):
        """Сценарий 3: отмена занятия."""
        self.schedule.add_class(FitnessClass("Йога", "Иванова А.", self.tomorrow,
10))
        self.schedule.cancel_class("Йога")
        titles = [c.title for c in self.schedule.classes]
        self.assertNotIn("Йога", titles)


if __name__ == "__main__":
    unittest.main()
```

Запустив файл с тестами, можно убедиться, что все они успешно пройдены (рисунок 2).

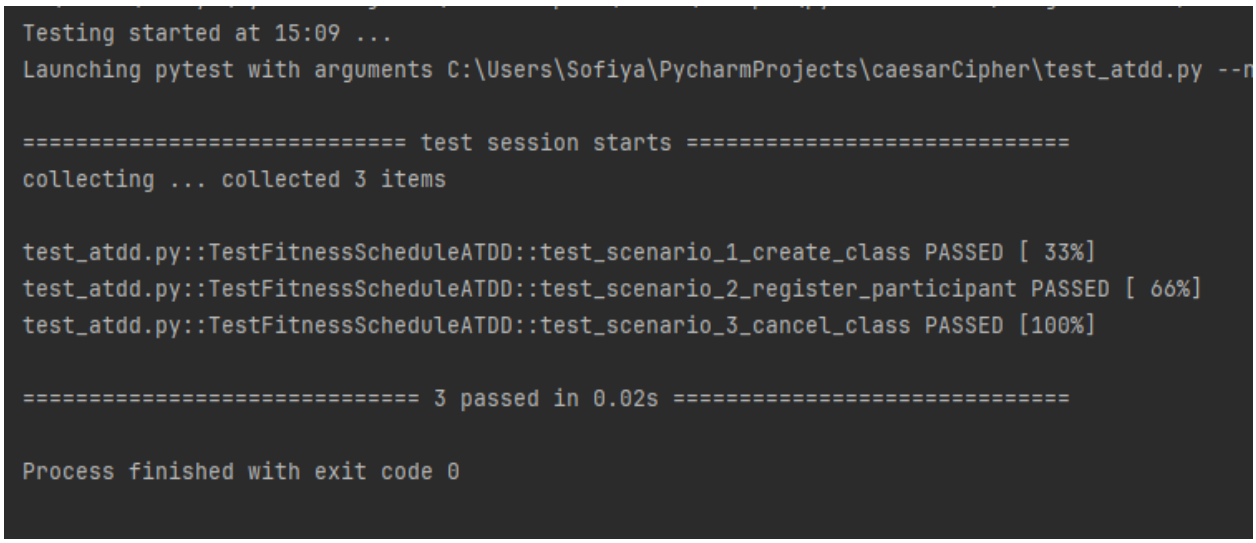


Рисунок 2. Тестирование

Этап №3. Реализация с помощью BDD (Behavior-Driven Development).

Созданы сценарии на понятном для всех участников проекта языке (Gherkin), которые описывают поведение системы (листинг 4).

Листинг 4. Сценарии на языке Gherkin

```
# language: ru
Функция: Расписание фитнес-классов
  Как администратор фитнес-клуба
  Я хочу управлять расписанием и регистрацией участников
  Чтобы клиенты могли записываться на занятия

  Сценарий: Создание нового занятия
    Дано расписание пустое
    Когда я добавляю занятие "Йога" с тренером "Иванова А." и вместимостью 10
    Тогда в расписании появляется занятие "Йога"

  Сценарий: Успешная регистрация участника
    Дано в расписании есть занятие "Йога" с вместимостью 10
    Когда участник "Петров П." записывается на "Йога"
    Тогда "Петров П." отображается в списке участников занятия "Йога"

  Сценарий: Регистрация на заполненное занятие
    Дано в расписании есть занятие "Пилатес" с вместимостью 1
    И участник "Петров П." уже записан на "Пилатес"
    Когда участник "Смирнов С." пытается записаться на "Пилатес"
```

```
    Тогда система возвращает ошибку "нет свободных мест"

  Сценарий: Отмена занятия
    Дано в расписании есть занятие "Йога"
    Когда я отменяю занятие "Йога"
    Тогда занятие "Йога" исчезает из расписания
```

Реализация сценариев была автоматизирована с помощью Behave для Python (листинг 5).

Листинг 5.Автоматизация с помощью Behave

```
from behave import given, when, then
from datetime import datetime, timedelta
from fitness import FitnessSchedule, FitnessClass, RegistrationError


@given("расписание пусто")
def step_empty_schedule(context):
    context.schedule = FitnessSchedule()


@given('в расписании есть занятие "{title}" с вместимостью {capacity:d}')
def step_class_exists(context, title, capacity):
    if not hasattr(context, "schedule"):
        context.schedule = FitnessSchedule()
    context.schedule.add_class(
        FitnessClass(title, "Тренер", datetime.now() + timedelta(days=1), capacity)
    )


@given('в расписании есть занятие "{title}"')
def step_class_exists_simple(context, title):
    step_class_exists(context, title, 10)


@given('участник "{participant}" уже записан на "{title}"')
def step_participant_registered(context, participant, title):
    context.schedule.register(title, participant)


@when('я добавляю занятие "{title}" с тренером "{trainer}" и вместимостью {capacity:d}')
def step_add_class(context, title, trainer, capacity):
    context.schedule.add_class(
```

```
        FitnessClass(title, trainer, datetime.now() + timedelta(days=1), capacity)
    )


@when('участник "{participant}" записывается на "{title}"')
def step_register(context, participant, title):
    context.schedule.register(title, participant)


@when('участник "{participant}" пытается записаться на "{title}"')
def step_try_register(context, participant, title):
    try:
        context.schedule.register(title, participant)
        context.error = None
    except RegistrationError as e:
        context.error = str(e)


@when('я отменяю занятие "{title}"')
def step_cancel(context, title):
    context.schedule.cancel_class(title)


@then('в расписании появляется занятие "{title}"')
def step_check_class_added(context, title):
    titles = [c.title for c in context.schedule.classes]
    assert title in titles, f"Занятие {title!r} не найдено. Список: {titles}"


@then('"{participant}" отображается в списке участников занятия "{title}"')
def step_check_participant(context, participant, title):
    fitness_class = context.schedule.get_class(title)
    assert participant in fitness_class.participants


@then('система возвращает ошибку "{message}"')
def step_check_error(context, message):
    assert context.error is not None, "Ожидалась ошибка, но её не было"
    assert message in context.error, f"Ожидалось {message!r}, получено
{context.error!r}"


@then('занятие "{title}" исчезает из расписания')
def step_check_class_removed(context, title):
    titles = [c.title for c in context.schedule.classes]
```

```
    assert title not in titles, f"Занятие {title!r} всё ещё в расписании"
```

Запустим сценарии (рисунок 3).

```
(.venv) PS C:\Users\Sofiya\PycharmProjects\caesarCipher> behave
USING RUNNER: behave.runner:Runner
Функция: Расписание фитнес-классов # features/fitness.feature:2
  Как администратор фитнес-клуба
  Я хочу управлять расписанием и регистрацией участников
  Чтобы клиенты могли записываться на занятия
  Сценарий: Создание нового занятия                                    # features/fitness.feature:7
    Дано расписание пусто                                              # features/steps/fitness_step
s.py:6 0.001s
    Когда я добавляю занятие "Йога" с тренером "Иванова А." и вместимостью 10 # features/steps/fitness_step
s.py:30 0.000s
    Тогда в расписании появляется занятие "Йога"                       # features/steps/fitness_step
s.py:56 0.001s

    Дано в расписании есть занятие "Йога" с вместимостью 10          # features/steps/fitness_steps.py:11
    Когда участник "Петров П." записывается на "Йога"                # features/steps/fitness_steps.py:37
    Тогда "Петров П." отображается в списке участников занятия "Йога" # features/steps/fitness_steps.py:62
0.000sгда "Петров П." отображается в списке участников занятия "Йога" # features/steps/fitness_steps.py:62

    Дано в расписании есть занятие "Пилатес" с вместимостью 1    # features/steps/fitness_steps.py:11 0.001
    И участник "Петров П." уже записан на "Пилатес"              # features/steps/fitness_steps.py:25 0.000
    Когда участник "Смирнов С." пытается записаться на "Пилатес" # features/steps/fitness_steps.py:42 0.001
    Тогда система возвращает ошибку "нет свободных мест"        # features/steps/fitness_steps.py:68 0.000
s   Тогда система возвращает ошибку "нет свободных мест"        # features/steps/fitness_steps.py:68

    Тогда занятие "Йога" исчезает из расписания # features/steps/fitness_steps.py:74 0.000s
    Тогда занятие "Йога" исчезает из расписания # features/steps/fitness_steps.py:74
1 feature passed, 0 failed, 0 skipped
4 scenarios passed, 0 failed, 0 skipped
13 steps passed, 0 failed, 0 skipped
Took 0min 0.007s
```

Рисунок 3. Запуск сценариев

Этап №4. Реализация с помощью SDD (Specification by Example).

В проекте созданы спецификации требований с использованием конкретных примеров, которые затем были преобразованы в автоматизированные тесты.

Спецификации для подхода SDD описаны в виде таблиц 1-3.

Таблица 1. Создание занятия

Название	Тренер	Вместимость	Ожидаемый результат
Йога	Иванова А.	10	Занятие в

			расписании
Пилатес	Сидорова М.	5	Занятие в расписании
Бокс	Орлов В.	0	Занятие без мест

Таблица 2. Регистрация участников

Занятие	**Вместимость**	**Уже записаны**	**Новый участник**	**Ожидаемый результат**
Йога	10	–	Петров П.	Успешно
Пилатес	1	Петров П.	Смирнов С.	Ошибка: нет свободных мест
Йога	10	Петров П.	Петров П.	Ошибка: уже записан
Бокс	5	–	Иванов И.	Успешно

Таблица 3. Отмена занятия

Занятие	**Статус до отмены**	**Ожидаемый результат**
Йога	существует	Удалено из расписания
Пилатес	не существует	Ошибка: не найдено

На основе спецификаций создадим автоматизированные тесты (листинг 6).

Листинг 6. Автоматизированные тесты

```
import unittest
from datetime import datetime, timedelta
from fitness import FitnessSchedule, FitnessClass, RegistrationError


class TestFitnessScheduleSDD(unittest.TestCase):
    """Тесты, построенные по таблицам спецификаций."""
```

```
    def setUp(self):
        self.schedule = FitnessSchedule()
        self.tomorrow = datetime.now() + timedelta(days=1)

    # --- Таблица 1: создание занятий ---
    def test_sdd_create_yoga(self):
        self.schedule.add_class(FitnessClass("Йога", "Иванова А.", self.tomorrow,
10))
        self.assertEqual(len(self.schedule.classes), 1)
        self.assertEqual(self.schedule.classes[0].capacity, 10)

    def test_sdd_create_pilates(self):
        self.schedule.add_class(FitnessClass("Пилатес", "Сидорова М.",
self.tomorrow, 5))
        self.assertEqual(self.schedule.get_class("Пилатес").capacity, 5)

    def test_sdd_create_box_zero_capacity(self):
        self.schedule.add_class(FitnessClass("Бокс", "Орлов В.", self.tomorrow, 0))
        self.assertEqual(self.schedule.get_class("Бокс").capacity, 0)

    # --- Таблица 2: регистрация ---
    def test_sdd_register_success(self):
        self.schedule.add_class(FitnessClass("Йога", "Иванова А.", self.tomorrow,
10))
        self.schedule.register("Йога", "Петров П.")
        self.assertIn("Петров П.", self.schedule.get_class("Йога").participants)

    def test_sdd_register_full(self):
        self.schedule.add_class(FitnessClass("Пилатес", "Сидорова М.",
self.tomorrow, 1))
        self.schedule.register("Пилатес", "Петров П.")
        with self.assertRaises(RegistrationError):
            self.schedule.register("Пилатес", "Смирнов С.")

    def test_sdd_register_duplicate(self):
        self.schedule.add_class(FitnessClass("Йога", "Иванова А.", self.tomorrow,
10))
        self.schedule.register("Йога", "Петров П.")
        with self.assertRaises(RegistrationError):
            self.schedule.register("Йога", "Петров П.")

    def test_sdd_register_box(self):
        self.schedule.add_class(FitnessClass("Бокс", "Орлов В.", self.tomorrow, 5))
        self.schedule.register("Бокс", "Иванов И.")
        self.assertIn("Иванов И.", self.schedule.get_class("Бокс").participants)
```

```
    # --- Таблица 3: отмена ---
    def test_sdd_cancel_existing(self):
        self.schedule.add_class(FitnessClass("Йога", "Иванова А.", self.tomorrow,
10))
        self.schedule.cancel_class("Йога")
        self.assertEqual(len(self.schedule.classes), 0)

    def test_sdd_cancel_missing(self):
        with self.assertRaises(RegistrationError):
            self.schedule.cancel_class("Пилатес")


if __name__ == "__main__":
    unittest.main()
```

Запустим тесты (рисунок 4).

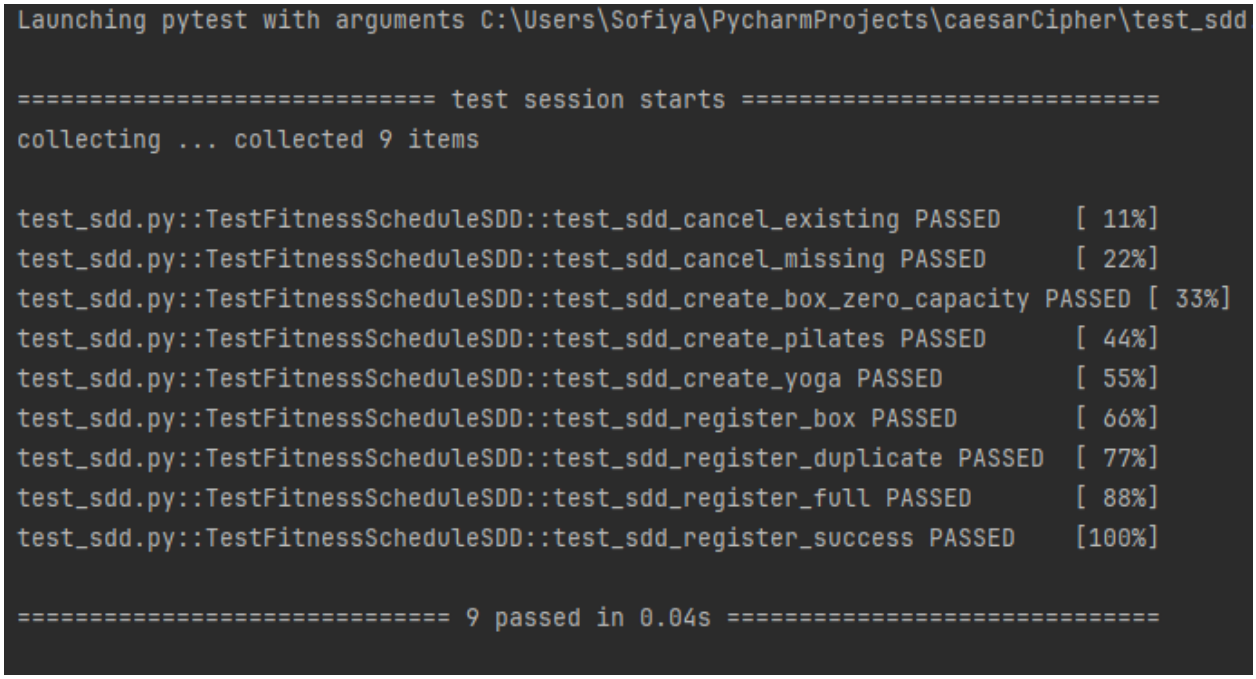


Рисунок 4. Тестирование

Анализ:

Каждый из представленных выше подходов имеет свои плюсы и минусы. TDD дает быструю обратную связь и защищает от регрессий на уровне кода, но требует времени на написание тестов. ATDD гарантирует, что продукт решает задачи заказчика, а BDD улучшает коммуникацию с помощью сценариев. Но их сложно поддерживать при частой смене

требований. SDD устраняет неоднозначности через конкретные примеры, но требует участия аналитиков.

Вывод:

В ходе практической работы на примере приложения «Расписание фитнес классов» были реализованы все четыре подхода к разработке через тестирование:

TDD – написаны 6 юнит-тестов, реализован минимальный код, проведен рефакторинг.

ATDD – составлены 3 приемочных сценариев.

BDD – созданы 4 сценария и их автоматизация.

SDD – построены 3 таблицы спецификаций с конкретными примерами и 9 автоматизированных тестов.

Итог: все юнит-тесты и сценарии прошли успешно. Комплексное применение подходов повышает качество, надежность и поддерживаемость кода, а также улучшает взаимодействие между разработчиками, тестировщиками и бизнесом.